Contents

Morpheus: On-Device Predictive Autocompletion for Basque Using State Space Models

Preprint — July 2026

Author: Xabier Ezpeleta

Code & Models: `github.com/itzune/morpheus`

Keywords: predictive autocompletion, Basque, Euskara, Mamba, State Space Models, agglutinative languages, on-device inference, keystroke savings, next-word prediction, subword tokenization

---

Abstract

Can a Basque text-editor autocompletion system run locally on a consumer device? We answer affirmatively by training **Morpheus**, a 91M-parameter Mamba-2 State Space Model on a 4.62B-token curated Basque corpus (~10B tokens seen). The model fits in 55 MB (Q4_K_M), runs at 318 tok/s on a 2017 laptop CPU with 97 ms end-to-end latency, and achieves 25.3% Character Savings Rate with 76% morpheme boundary accuracy.

A head-to-head comparison against **Latxa 8B (base)** establishes a **two-tier deployment architecture fixed by hardware**: Morpheus is the only model that runs on the edge (40.7 tok/s on a consumer CPU); Latxa 8B is the server-side quality ceiling (+8.35 CSR points) but collapses to 2.8 tok/s on the same hardware. Along the way, we expose a **fertility paradox** — lower tokenizer fertility destroys morphological accuracy in agglutinative languages — and a **CSR paradox** — the keystroke-savings metric structurally penalizes the very language the system is designed to serve. We also contribute five inference engineering strategies that add 3.9x CSR on top of the raw model, and a Fill-in-the-Middle extension for cursor-mid-text completion.

---

1. Introduction

Basque (Euskara) is a low-resource, morphologically agglutinative language isolate. A single verb can encode subject, object, indirect object, tense, mood, and aspect through suffix chains (e.g., *ikusiko zenizkidakeen* — "you would have been able to see them to me"). Nouns take 12+ case suffixes plus number and definiteness marking. Predicting the next word therefore requires **morphological productivity** — the ability to generate grammatically correct suffix sequences never seen as a unit during training.

Production autocompletion systems fall into three paradigms: **server-side multi-token completion** (Gmail Smart Compose, GitHub Copilot), **on-device next-word prediction** (Gboard), and **on-device multi-token continuation** — a Smart Compose equivalent for desktop editors. This third paradigm is the gap: no system runs it on-device for a morphologically complex language. None of the three targets Basque.

Two strategic paths present themselves. **Adapting an existing Basque LLM** (the HiTZ Latxa family, Llama-3.1-based) is viable for the server tier, but at 8B parameters it cannot serve the on-device case. **Training a new architecture from scratch** is necessary for the edge tier. We selected Mamba-2 — a State Space Model with O(1) per-step inference and constant memory, the same property Google exploited for Smart Compose but solved at the architecture level rather than with data-center TPUs.

System	Params	Size	Deployment	Architecture	Paradigm
Gboard (on-device NWP)	1.4M	1.4 MB	On-device (mobile)	LSTM	Next-word
Gmail Smart Compose	~80M	Server	Cloud TPU	LSTM	Multi-token
GitHub Copilot	Multi-B	Server	Cloud GPU	Transformer (FIM)	Multi-token
Morpheus	**91M**	**55 MB**	**On-device (laptop)**	**Mamba-2**	**Multi-token**
Latxa 8B (base)	8B	6.6 GB	Server (GPU)	Transformer	Multi-token

Table 1. Morpheus in context. Both Google and Morpheus chose recurrent architectures to avoid KV-cache latency; Morpheus achieves it without the data center.

---

2. Architecture Selection

Our constraints demand <=300 MB on disk, P90 <=50 ms per-token latency, zero network calls, and training on a single NVIDIA L40 GPU. We evaluated three candidates:

Criterion (weight)	xLSTM	Distilled Transformer	**Mamba-2**
Prediction quality (30%)	3	**5**	4

Criterion (weight)	xLSTM	Distilled Transformer	**Mamba-2**
Inference latency (25%)	**5**	3	**5**
Engineering risk (20%)	**4**	2	3
Future-proofing (10%)	3	4	**5**
Weighted Score	3.80	3.70	**4.05**

Mamba-2 combines LSTM-like inference properties (constant memory, no KV cache) with substantially better language modeling capacity. The distillation path was rejected because Gemma's 256K vocabulary would consume the entire parameter budget in embeddings alone, and KV cache on consumer CPU violates the latency constraint.

---

3. The Tokenizer Decision: Why 4K Vocabulary

The tokenizer is the single most consequential design decision for agglutinative language modeling. Our research synthesized six recent papers (2024–2026) covering 70+ languages, converging on three findings: (1) fertility is misleading for agglutinative tokenizers — low fertility means fused morphemes; (2) Unigram outperforms BPE for agglutinative languages; (3) smaller vocabularies preserve morpheme boundaries.

We trained SentencePiece Unigram tokenizers at 4K, 8K, 16K, and 32K and measured **MorphAcc consistency** — the percentage of test words where the tokenizer places a boundary at the known morpheme split:

Vocab Size	Fertility	MorphAcc
4,000	2.58	**66.7%**
8,000	2.28	61.9%
16,000	2.06	52.4%
32,000	1.85	28.6%

This replicates the QuechuaTok finding (Contreras, 2026) for a different language family. The degradation is monotonic and accelerating: each vocabulary doubling costs progressively more morphological accuracy. This is the **fertility paradox**: lower fertility (fewer tokens per word) is achieved *exactly by* fusing morphemes into opaque units. The 32K tokenizer memorizes *etxetik* ("from the house") as one token; the 4K tokenizer splits it as `▁etxe tik` — root and ablative suffix independently accessible for recombination.

The contrast is most stark in verbal morphology. At 4K, the pluralizer `zki` is an independent reusable token across *dizkizut*, *dakizkioke*, *zitzaizkidan*. At 32K, these are opaque atomic tokens sharing no subword structure. At 91M parameters, the model cannot afford a suboptimal tokenizer — and at 4K, only 3.4% of parameters are embeddings (vs. 27% at 32K), freeing capacity for the SSM layers.

---

4. Training

Parameter	Value
Architecture	Mamba-2 (pure SSM), d_model=768, 24 layers
Total parameters	~91M
Vocabulary	4,000 (SentencePiece Unigram)
Corpus	4.62B tokens, curated from Latxa Corpus v2 (11 of 14 sub-corpora)
Total tokens seen	~10B (~2.16 epochs)
Hardware	1x NVIDIA L40 (48 GB)
Best checkpoint	Step 74K — PPL 7.13 (converged, flat from step 67K)

Data curation was critical. We omitted 3 of 14 sub-corpora — `hplt-v1` (83.8% duplicates), `BOG` (sentence-split legal fragments), and `Aldizkariak` (35% boilerplate) — and applied a four-phase cleaning pipeline (re-parsing, normalization, content filtering, deduplication). An LLM-based audit rated retained sources 4.6/5. Validation leakage prevention excluded 68,755 held-out lines from training.

Convergence analysis revealed that the model effectively stopped learning at ~8.8B tokens — the final ~1.2B tokens produced no measurable PPL improvement. The improvement rate dropped 8.8x between the first and second halves of training. With ~20–30% corpus noise, the effective high-quality data is ~7–8B tokens, matching the convergence point. **For low-resource languages, data quality is the binding constraint, not quantity** — a 2–5B token high-quality corpus would likely match the current 10B mixed-quality one.

---

5. Inference Engineering for Agglutinative Keyboards

Deploying a subword language model as a real-time keyboard with whole-word suggestion chips exposes the **tokenization trap** — a structural failure mode invisible in batch evaluation. The word *Kaixo* ("hello") tokenizes as `[▁Ka, i, xo]`, but when the user types *Kaix*, the tokenizer produces `[▁Ka, ix]` — a different path that cannot reach the correct completion. This is not a model deficiency; it is a property of subword tokenization, especially acute in agglutinative languages with long, morphologically complex words.

Five strategies address this:

1. **Retokenization fallback.** Query the model from progressively shorter prefixes in parallel, filter results by the user's typed prefix. For *Kaix*, the system also queries from *Kai* and *Ka* — the latter reaches the correct token path and surfaces *Kaixo*.

2. **Sticky merge (candidate carry-forward).** When the model predicts *izan* after *idatzia* and the user types *i*, the prediction vanishes because the token path changes. A sticky pool preserves previous candidates, filtering by the current prefix and boosting survivors.

3. **Top-k exceeds display-k.** The keyboard shows 3 chips but fetches 5, populating the sticky pool with lower-ranked but relevant predictions.

4. **Next-word candidate extraction.** When greedy continuation begins with a space-prefixed token, extract it as a next-word candidate rather than discarding it.
5. **Completion logging with replay.** Every chip acceptance is logged as a (context, candidates, accepted word) tuple. This transforms real sessions into an evaluation dataset replayable against any checkpoint.

Impact: These strategies add **3.9x CSR** on top of the raw model (0.094 -> 0.362), demonstrating that inference engineering is a major contribution, not a marginal optimization.

---

6. Evaluation and Results

6.1 Core Metrics

Metric	Result	Interpretation
Held-out PPL	7.13	Strong LM quality; converged
CSR (macro, n=300)	25.26% [24.0%, 26.5%]	Meaningful keystroke savings
MorphAcc	76%	3.8x improvement over 32K tokenizer
BPC	0.970	Matches GPT-2 eus-euscrawl (0.981) at 27% fewer params
Q4_K_M size	55 MB	Well within 300 MB budget

PPL is the only reliable checkpoint-ranking metric. It improved monotonically across all 14 evaluation files (7.56 -> 7.17 -> 7.13). CSR, MorphAcc, and case paradigm metrics all saturated or produced noisy, non-monotonic signal. CSR is fragile to implement (string prompts gave ~4% vs ~28% with token-ID prompts due to BOS/tokenizer bugs) and cannot distinguish checkpoints at n=300 (all confidence intervals overlap).

6.2 Cross-Model Comparison

Bits Per Character (BPC) is the correct tokenizer-independent metric for comparing models with different vocabularies:

Model	Params	BPC	CSR (simplified)	Word Accuracy
GPT-2 eus-euscrawl	124M	0.981	0.110	37.6%
Morpheus (Mamba-2)	**91M**	**0.970**	**0.094**	**60.4%**
Latxa 8B (base)	8B	—	**0.332**	—

Morpheus matches GPT-2's BPC at fewer parameters (the difference is primarily attributable to 11x more training data). **The Latxa 8B comparison** fixes the deployment architecture: Latxa saves +8.35 CSR points with artifact-free output, but is GPU-bound. On the consumer laptop CPU, Latxa collapses to 2.8 tok/s (2,869 ms/request, 19x over the latency budget), while Morpheus sustains 40.7 tok/s.

Hardware	Model	Latency	tok/s	Memory
L40 (GPU)	Morpheus Q5_K_M	75 ms	106	602 MiB VRAM
L40 (GPU)	Latxa 8B Q6_K	115 ms	69.5	6,988 MiB VRAM
i7-8550U (CPU)	Morpheus Q5_K_M	196 ms	40.7	266 MiB RAM
i7-8550U (CPU)	Latxa 8B Q6_K	2,869 ms	2.8	6,648 MiB RAM

The qualitative difference is clear: Latxa commits to semantically specific continuations (a concrete meeting time, an encryption property), while Morpheus often drifts into high-frequency connective filler or unrelated statistical patterns. Morpheus's sweet spot is **formulaic completion** — email openings, fixed collocations, administrative phrasing — and **domain-specialized fine-tunes**.

6.3 The CSR Paradox

A typing simulation with 15 parallel sentences (5 per language, identical semantic content) revealed that the model's native Basque achieves the **lowest** simulated CSR:

Language	Top-3 Accuracy	Simulated CSR	Avg. prefix before acceptance
Basque (native)	**79.5%**	**19.6%**	**4.4 chars**
English (<1% corpus)	72.0%	25.5%	2.7 chars
Spanish (<1% corpus)	73.9%	29.3%	3.1 chars

This is not a model deficiency — it is a structural property of agglutinative morphology. Basque words are longer, so the user must type more characters before the model can confidently predict the correct form. A word like *paseatzera* ("to go for a walk") cannot be predicted from *pa*; the model needs *paseatzer* before converging. Meanwhile, English *walk* is predictable from *w* given sufficient context. Basque achieves the **highest** Top-3 accuracy (79.5%) — the model identifies the correct word more often — but the keystroke savings are consumed by the longer prefix.

CSR penalizes the very language such systems are designed to serve and should not be used as a primary optimization target. This aligns with GitHub's finding that acceptance-rate optimization "could lead to incorrectly favoring a high volume of simple and short suggestions" (Fu & Mogensen, 2025).

---

7. Fill-in-the-Middle (FIM) Extension

The AR-only model can only extend a prefix. For desktop text editing, the cursor often sits within a sentence, requiring text that bridges what precedes and follows — the **Fill-in-the-Middle** objective. We extended Morpheus via continued pre-training from the step-74K checkpoint, using Code Llama-style FIM tokens (<PRE>, <SUF>, <MID>, <EOT>), token-level splitting, and a 500M-token budget.

The key engineering finding is the stop-token reliability problem. A naive 50/50 FIM/AR mix produces coherent infill but fails to reliably emit `<EOT>` (~77% emission), causing over-generation and deeply negative keystrokes saved (~-25%). The root cause is signal sparsity — `<EOT>` is a single token per FIM example. A **5x loss weight on `<EOT>`** with a **70/30 FIM/AR ratio** resolves this:

Metric	Before	After (5x EOT weight)
`<EOT>` emission	~77%	88.4%
Keystrokes saved	-25%	-5.9%
AR valid PPL	7.13	7.5 (stable)
Premature truncation	—	1.4% (< 15% threshold)

The feared premature-truncation failure mode did not materialize. AR capability is preserved ("FIM-for-free"), and the model now slightly under-generates (40.3 vs. 45.0 chars) — a preferable failure mode for users.

---

8. Known Limitations

- **CSR of 25%** is below the ~80% achievable in English autocomplete — partly model quality, partly the structural CSR paradox.
- **Corpus-induced artifacts:** The model over-predicts dates and numbers because the corpus is dominated by encyclopedic and journalistic prose. *Aipatu bezala,* -> *2015eko ekainean,* instead of a general continuation. This is domain mismatch: Smart Compose avoids it by training on emails and deploying for emails; for Basque, no large conversational corpus exists.
- **No morphological pre-segmentation** yet: MorphAcc could improve from 76% toward 83%+ with Apertium-based surface-preserving boundaries.
- **No user study:** All evaluation is simulation-based. The model is too large for mobile (Gboard: 1.4M/1.4 MB); a distilled ~5–10M variant has not been trained.
- **Evaluation limitations:** PPL is the only reliable metric; CSR and MorphAcc saturate at available sample sizes. The metric inversion (Section 6.3) — autocomplete metrics moving opposite to PPL improvement — suggests that a better model distributes probability across valid morphological variants, depressing exact-match accuracy.

---

9. Conclusion

Morpheus demonstrates that **on-device predictive autocompletion for an agglutinative language is feasible**. A 91M Mamba-2 model, fitting in 55 MB with zero network calls, provides real-time ghost-text completion on a 2017 laptop CPU — comparable in scale to Gmail Smart Compose but without the data-center dependency.

Key Contributions

1. **The fertility paradox.** For agglutinative tokenizers, lower fertility *destroys* morphological accuracy. MorphAcc drops from 66.7% (4K) to 28.6% (32K) — replicating QuechuaTok across language families.

2. **The CSR paradox.** Keystroke-savings metrics structurally penalize morphologically complex languages. The model's native Basque achieves the *lowest* CSR despite the *highest* Top-3 accuracy. CSR should never be used as a primary optimization target for agglutinative autocomplete.

3. **Inference engineering as a first-class contribution.** Five strategies addressing the tokenization trap add 3.9x CSR on top of the raw model — retokenization fallback, sticky merge, top-k exceeding display-k, next-word extraction, and completion logging with replay.

4. **Two-tier deployment architecture.** Morpheus (91M, 55 MB) runs on the edge for formulaic completion and domain fine-tunes. Latxa 8B (base) is the server-side quality ceiling (+8.35 CSR points, cross-domain competence), but GPU-bound. The split is a hardware constraint, not a preference.

5. **Data quality over quantity.** The model converged at ~8.8B tokens; a 2–5B token high-quality corpus would likely match the full 10B mixed-quality one. For low-resource languages, aggressive quality filtering dominates raw scale.

6. **FIM stop-token engineering.** The `<EOT>` signal is too sparse for reliable learning within modest CPT budgets. A 5x loss weight resolves over-generation without inducing premature truncation.

The Path Forward

The model has converged. The next steps are: (1) integrate Apertium Basque for morpheme pre-segmentation; (2) distill a mobile-variant model (~5–10M); (3) run FIM continued pretraining on Latxa 8B (base) as the server-side model; (4) domain-specific fine-tuning to mitigate corpus-induced artifacts; and (5) conduct user studies with real Basque speakers.

References

1. Contreras, M. (2026). *QuechuaTok: Morphological Boundary Accuracy as a Necessary Metric for Tokenizer Evaluation in Agglutinative Low-Resource Languages.* arXiv:2606.23943.
2. Chen, M. X., et al. (2019). *Gmail Smart Compose: Real-Time Assisted Writing.* arXiv:1906.00080.
3. Fu, S., & Mogensen, J. (2025). *The Road to Better Completions: Building a Faster, Smarter GitHub Copilot.* GitHub Blog.
4. Gu, A., & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv:2312.00752.
5. Dao, T., & Gu, A. (2024). *Transformers are SSMs: Generalized Models and Efficient Algorithms through Structured State Space Duality.* arXiv:2405.21060.
6. Etxaniz, J., et al. (2024). *Latxa: An Open Language Model and Evaluation Suite for Basque.* arXiv:2403.20266.
7. Bavarian, M., et al. (2022). *Efficient Training of Language Models to Fill in the Middle.* arXiv:2207.14255.
8. Trnka, K., & McCoy, K. (2008). *Evaluating Word Prediction: Framing Keystroke Savings.* ACL 2008.
9. Lane, W., Harrigan, A., & Arppe, A. (2022). *Interactive Word Completion for Plains Cree.* ACL 2022.

10. Xu, N., & Kim, A. (2026). *Tokenization and Morphological Fidelity in Uralic NLP*. arXiv.
11. Stephen, A., & Libovický, J. (2026). *Evaluating Morphological Plausibility of Subword Tokenization.* arXiv.
12. Hoffmann, J., et al. (2022). *Training Compute-Optimal Large Language Models* (Chinchilla). arXiv:2203.15556.
13. Sardana, N., et al. (2024). *Beyond Chinchilla-Optimal: Accounting for Inference in Language Model Scaling Laws.* arXiv:2401.00448.
14. Roziere, B., et al. (2023). *Code Llama: Open Foundation Models for Code.* arXiv:2308.12950.

Current as of July 2026 — AR training complete (76,294 steps, best checkpoint step 74,000, PPL 7.13); FIM continued pre-training complete.